

Browser Fingerprinting Lab Report

M235 Information Security | UM6P | 4 October 2026

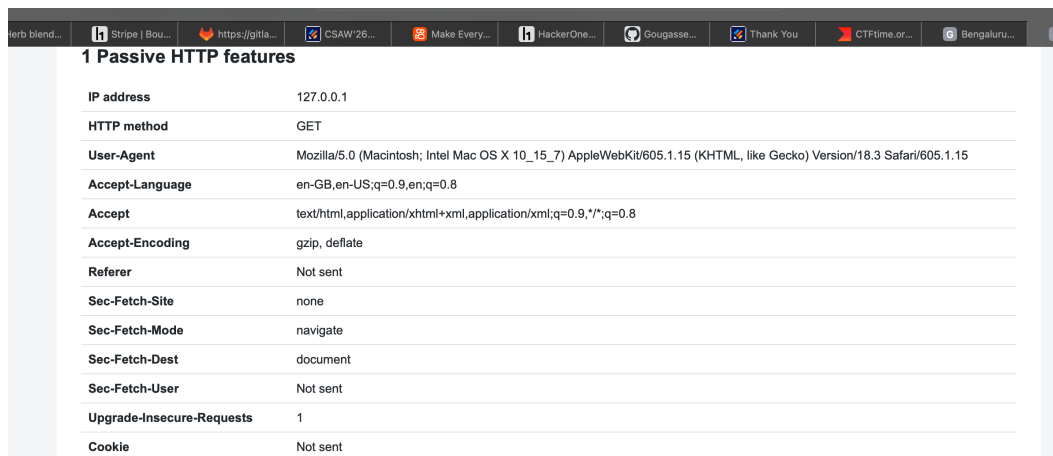
Aim. I tested what a website can learn about a browser without storing a cookie. I compared Safari normal mode, Safari Private Browsing, and Chrome Incognito. The lab server ran locally with Flask at 127.0.0.1:8000.

1. Passive fingerprinting

Passive clues arrive with the HTTP request. **User-Agent** can show the stated browser and operating system, **Accept-Language** preferred languages, and **Accept** and **Accept-Encoding** supported formats. **Referer** may show the previous page; **Sec-Fetch** headers describe request context.

Header	What it can show	Safari normal
User-Agent	Browser, engine, OS	Safari 18.3, Mac OS X
Accept-Language	Language preference	en-GB, en-US, en
Accept / Accept-Encoding	Formats and compression	HTML, XML; gzip, deflate
Sec-Fetch-Site / Mode / Dest	Request context	none / navigate / document
Referer / Cookie	Previous page / stored state	Not sent on direct visit

The observed header combination can help distinguish browser setups. Fetch metadata mostly describes the request rather than a person. No Cookie header was sent on these direct visits. Safari Private Browsing kept the listed passive values unchanged; Chrome reported a different user agent and language list.



1 Passive HTTP features	
IP address	127.0.0.1
HTTP method	GET
User-Agent	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.3 Safari/605.1.15
Accept-Language	en-GB,en-US;q=0.9,en;q=0.8
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding	gzip, deflate
Referer	Not sent
Sec-Fetch-Site	none
Sec-Fetch-Mode	navigate
Sec-Fetch-Dest	document
Sec-Fetch-User	Not sent
Upgrade-Insecure-Requests	1
Cookie	Not sent

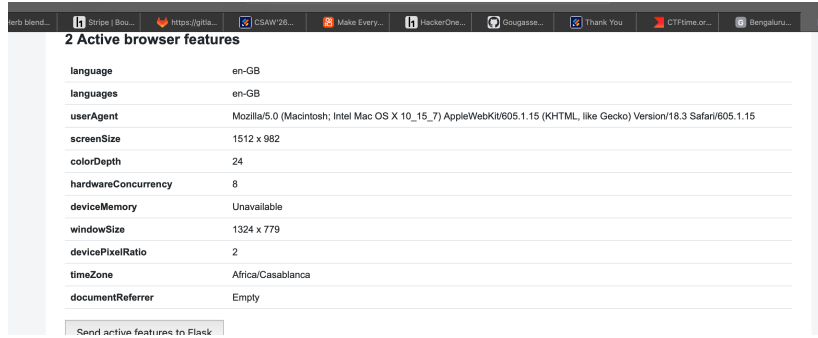
Figure 1: HTTP request headers shown by the lab.

2. Active feature collection

The page read browser and screen properties with JavaScript, displayed them, and sent the values to Flask. Processor count, screen size, language, memory and time zone may help distinguish environments. Some values can be unavailable or adjusted by privacy settings.

Feature	Safari normal	Safari private	Chrome Incognito
Language	en-GB	en-GB	fr-FR
Screen size	1512 × 982	1324 × 807	1512 × 982
Colour depth	24	24	30
Logical processors	8	8	14
Memory	unavailable	unavailable	16
Pixel ratio	2	2	2
Time zone	Africa/Casablanca	Africa/Casablanca	Africa/Casablanca

Safari Private Browsing reported a viewport-sized screen value, while language, processor count and time zone remained the same. Chrome exposed different browser and language values. Private browsing changed some exposed data but did not remove all browser clues.



2 Active browser features

language	en-GB
languages	en-GB
userAgent	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.3 Safari/605.1.15
screenSize	1512 x 982
colorDepth	24
hardwareConcurrency	8
deviceMemory	Unavailable
windowSize	1324 x 779
devicePixelRatio	2
timeZone	Africa/Casablanca
documentReferrer	Empty

Send active features to Flask

Figure 2: Active feature values in Safari normal mode.

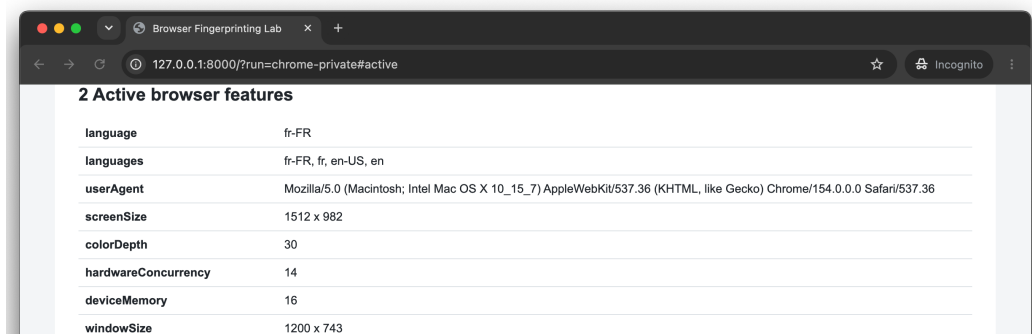


2 Active browser features

language	en-GB
languages	en-GB
userAgent	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.3 Safari/605.1.15
screenSize	1324 x 807
colorDepth	24
hardwareConcurrency	8
deviceMemory	Unavailable
windowSize	1324 x 807
devicePixelRatio	2
timeZone	Africa/Casablanca
documentReferrer	Empty

Send active features to Flask

Figure 3: Active feature values in Safari Private Browsing.



2 Active browser features

language	fr-FR
languages	fr-FR, fr, en-US, en
userAgent	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
screenSize	1512 x 982
colorDepth	30
hardwareConcurrency	14
deviceMemory	16
windowSize	1200 x 743

Figure 4: Active feature values in Chrome Incognito.

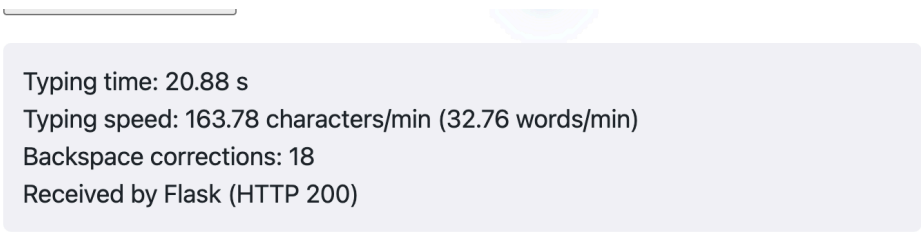
3. Typing behaviour

1. **Speed formula.** Elapsed time is measured in seconds. For a sentence of N characters typed in t seconds, characters per minute (CPM) is $N \times 60/t$. The conventional words-per-minute estimate is CPM divided by 5.

$$t = \frac{\text{finish} - \text{start}}{1000}, \quad \text{CPM} = \frac{N \times 60}{t}, \quad \text{WPM} = \frac{\text{CPM}}{5}.$$

2. **JavaScript reference used.** I used the handout's items for selecting page elements, handling key and input events, timing with `performance.now()`, displaying results, and sending JSON with `fetch()`. 3. **Algorithm.** Wait for the first character and record `performance.now()`; count each Backspace correction; after each input change compare the text with the target; once it matches, stop timing, calculate the speeds, display the values and send them to Flask. 4. **Implementation.** The required file is `static/fingerprint2.js`. It listens for `keydown` to start the timer and count Backspace, then `input` to test for completion. It displays the results and includes a Restart button to clear the sample. Straight and curly apostrophes are treated as equivalent so keyboard punctuation does not block completion. 5. **Request sent.** On completion the page posts JSON to `/collect`: `typing_time`, `typing_speed_cpm`, `typing_speed_wpm`, and `correction_count` (with the task name and character count). The page showed HTTP 200. 6. **Verification and sample output.** The Flask console logged a POST `/collect` with 57 characters, 20.882 seconds, 163.777 CPM, 32.755 WPM and 18 corrections, followed by HTTP 200. The browser result and server log agree. The displayed values are rounded to two decimals.

Typing time	20.88 s
Typing speed	163.78 characters/min (32.76 words/min)
Backspace corrections	18
Flask response	HTTP 200



Typing time: 20.88 s
Typing speed: 163.78 characters/min (32.76 words/min)
Backspace corrections: 18
Received by Flask (HTTP 200)

Figure 5: Completed typing test: result displayed on the page after Flask accepted the request.

This is one person's single run; typing speed and corrections vary between attempts, so one sample is not enough to identify someone.

Conclusion

A site can collect browser clues through request headers and JavaScript even when cookies are absent. Private browsing changed some exposed features but left others available. Typing timing adds another behavioural signal, though a single sample is not proof of identity.